

Fast PUF-Based Authentication Protocol for UAV Swarm Networks: Design, OMNeT++ Simulation, and Performance Evaluation

Md Ziyad Hussain

Department of Computer Science and Engineering

University Name

Email: md.ziyad.hussain@example.com

Abstract—Unmanned Aerial Vehicle (UAV) swarm networks demand lightweight, fast, and secure authentication protocols to enable real-time collaborative operations such as formation flying, search-and-rescue, and stadium aerial surveillance. This paper presents a comprehensive four-phase Physical Unclonable Function (PUF)-based authentication protocol that achieves mutual authentication between UAVs and a Ground Station (Phase 2), direct peer-to-peer authentication between UAV pairs without Ground Station involvement (Phase 3), and secure session key establishment (Phase 4). We implement and evaluate the protocol in OMNeT++ 6.3 with two hash modes: SHA3-256 (truncated to 160 bits) and SPONGENT-160, a lightweight hardware-oriented hash. Our simulation of a 10-UAV stadium surveillance scenario demonstrates Phase 2 authentication latency of 0.67 ms (SHA3) and 7.37 ms (SPONGENT), and Phase 3 peer authentication latency of 0.48 ms (SHA3) and 14.54 ms (SPONGENT) per pair, with 100% authentication success across all 45 full-mesh peer pairs. The protocol achieves 10–18 $\times$ faster authentication than RSA-2048 or ECC-256, with only 430 bytes total communication overhead. We provide detailed per-message timing breakdowns, hash-mode comparisons, and discuss projected hardware ASIC/FPGA performance where SPONGENT-160 achieves sub-10 μ s per hash within 2,190 gate equivalents.

Index Terms—UAV authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, swarm security, OMNeT++, lightweight cryptography, peer-to-peer authentication

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs) have become integral to modern defense, disaster response, precision agriculture, and public safety operations. In multi-UAV swarm deployments—such as stadium aerial surveillance with 10 or more drones providing perimeter security—the vehicles must authenticate each other rapidly to prevent unauthorized nodes from injecting false data or commands into the formation [1], [12].

Traditional public-key infrastructure (PKI) approaches using RSA-2048 or ECC-256 certificates incur authentication latencies of 8–15 ms per handshake on embedded ARM processors, which is unacceptable for safety-critical swarm coordination requiring sub-millisecond response times [2]. Furthermore, PKI requires secure key storage—a liability when a UAV is physically captured by an adversary.

Physical Unclonable Functions (PUFs) offer a compelling alternative by generating unique challenge-response pairs (CRPs) from intrinsic manufacturing variations in silicon

hardware [13]. Since PUFs derive keys from physics rather than stored secrets, they provide inherent resistance to physical attacks: cloning the PUF requires replicating sub-nanometer manufacturing variations, which is computationally and physically infeasible with current technology [9].

In this paper, we present a complete four-phase PUF-based authentication framework:

- 1) **Phase 1 (Enrollment)**: Secure registration of UAVs with the Ground Station, establishing CRP databases with BCH error correction helper data.
- 2) **Phase 2 (UAV-GS Authentication)**: Four-message mutual authentication between each UAV and the Ground Station upon network entry.
- 3) **Phase 3 (Peer-to-Peer Authentication)**: Three-message direct mutual authentication between any pair of UAVs without Ground Station involvement.
- 4) **Phase 4 (Session Key Establishment)**: Derivation of symmetric session keys for encrypted communication.

We implement and evaluate this protocol in OMNeT++ 6.3, a discrete event network simulator, with a realistic 10-UAV stadium surveillance scenario. Our contributions include:

- A complete four-phase PUF protocol with full-mesh Phase 3 peer authentication (45 pairs for 10 UAVs).
- OMNeT++ implementation with dual hash mode support (SHA3-256 and SPONGENT-160).
- Detailed per-message, per-phase timing analysis with separate initiator/responder breakdowns.
- Comparison with RSA-2048, ECC-256, and pre-shared key approaches.
- Analysis of projected hardware performance for SPONGENT-160 on FPGA/ASIC platforms.

The remainder of this paper is organized as follows: Section II surveys related work. Section III details the four-phase protocol design. Section IV describes the OMNeT++ implementation. Section V presents simulation results. Section VI compares with existing methods. Section VII discusses hardware implementation considerations. Section VIII provides security analysis, and Section X concludes.

II. RELATED WORK

A. PUF-Based Authentication for IoT and UAVs

PUF-based authentication has gained significant attention for resource-constrained devices. Gope and Sikdar [2] proposed a lightweight PUF-based authentication and key agreement protocol for smart UAV networks, achieving mutual authentication with minimal computational overhead. Their protocol requires only hash and XOR operations but does not address peer-to-peer authentication between UAVs.

Chatterjee et al. [12] presented a PUF-based secure anonymous authentication protocol for UAVs in cross-domain environments, focusing on emergency rescue and collaborative scenarios. Their scheme eliminates the need for a trusted third party during inter-domain authentication but introduces significant communication overhead.

For industrial wireless sensor networks, Dey and Hosain [5] proposed MITM- and DoS-resistant PUF authentication via sensor-initiated registration, addressing specific attack vectors relevant to UAV deployments. Their approach to sensor-initiated registration inspires our Phase 1 enrollment design.

B. Peer-to-Peer UAV Authentication

Direct peer-to-peer authentication between UAVs remains an open challenge. Bera et al. [3] proposed a secure and efficient peer-to-peer authentication-attestation protocol emphasizing low latency for real-time UAV control. Their protocol achieves sub-millisecond authentication but relies on pre-shared symmetric keys, which introduces key storage vulnerabilities.

Our protocol combines the PUF-based security of [2] with the peer-to-peer efficiency of [3], using Ground Station-distributed credentials to enable PUF-verified peer authentication without GS involvement during flight.

C. Lightweight Hash Functions

SPONGENT [22] is a family of lightweight sponge-based hash functions designed for hardware efficiency. SPONGENT-160 uses a 160-bit state (16-bit rate, 144-bit capacity) with 80 permutation rounds, achieving a gate count of approximately 2,190 GE—significantly smaller than SHA3/Keccak’s 38,000 GE. While SPONGENT is slow in software due to bit-level permutations, it is specifically optimized for ASIC/FPGA implementations where it achieves sub-10 μ s per hash [20].

SHA3-256 (Keccak) [23], in contrast, uses a 1600-bit state with 24 rounds optimized for 64-bit processors, achieving excellent software performance via OpenSSL’s hand-optimized assembly but at higher hardware cost.

D. Error Correction for PUFs

PUF responses exhibit noise due to environmental variations (temperature, voltage), requiring error correction. BCH codes are widely used as fuzzy extractors for PUFs [6], [7]. We employ BCH(255, 131, 18), which corrects up to 18 bit errors ($\approx 7\%$ BER), matching the noise characteristics of ring oscillator and arbiter PUFs [18].

TABLE I: Protocol Notation

Symbol	Description
UAV_i	i -th Unmanned Aerial Vehicle
ID_i	Permanent identity of UAV_i
TID_i	Temporary identity (for anonymity)
C_i	PUF challenge for UAV_i
$R_i = \text{PUF}_i(C_i)$	PUF response
$h(\cdot)$	Hash function (SPONGENT-160 or SHA3)
$\oplus$	XOR operation
$\parallel$	Concatenation
N_i	Random nonce generated by UAV_i
T_i	Timestamp
SK_{ij}	Session key between UAV_i and UAV_j
h_i	BCH helper data for error correction

III. PROTOCOL DESIGN

A. Notation

Table I summarizes the notation used throughout this paper.

B. Phase 1: Enrollment/Registration

Enrollment occurs in a secure facility with tamper-proof communication channels. For each UAV_i to be enrolled:

Step 1.1: PUF Interrogation. The GS generates $n \geq 12$ random challenges $\{C_i^{(1)}, C_i^{(2)}, \dots, C_i^{(n)}\}$ and sends each to UAV_i . The UAV evaluates its PUF:

$$R_i^{(k)} = \text{PUF}_i(C_i^{(k)}) \quad (1)$$

Step 1.2: Error Correction Setup. For each CRP, the GS generates BCH helper data:

$$h_i^{(k)} = \text{BCH_encode}(R_i^{(k)}) \oplus R_i^{(k)} \quad (2)$$

where BCH(255, 131, 18) can correct up to 18 bit errors ($\approx 7\%$ BER). The helper data is public information that does not reveal $R_i^{(k)}$ but enables reliable PUF response reconstruction under noise.

Step 1.3: Identity Assignment. The GS generates for each UAV_i :

- Permanent identity ID_i (64-bit unique identifier)
- Temporary identity $TID_i = h(ID_i \parallel \text{random_seed})$
- Registration token $\tau_i = h(R_i^{(1)} \parallel ID_i \parallel T_{\text{reg}})$

Step 1.4: Storage. The GS stores $\{ID_i, TID_i, \{(C_i^{(k)}, h_i^{(k)})\}_{k=1}^n\}$. The UAV stores only $\{TID_i, \tau_i, \text{GS_public_params}\}$. No raw PUF responses are stored—they are regenerated on demand.

C. Phase 2: UAV-Ground Station Authentication

Phase 2 is a four-message protocol executed when a UAV enters the operational field.

Message 1 (Authentication Request): $UAV_i \rightarrow \text{GS}$

$$\alpha_1 = h(TID_i \parallel T_1 \parallel N_1 \parallel \tau_i) \quad (3)$$

$$M_1 = (TID_i, T_1, N_1, \alpha_1) \quad (4)$$

Format: $64 + 32 + 128 + 160 = 384$ bits.

Message 2 (Challenge Issuance): GS $\rightarrow$ UAV_{*i*}

$$\beta_1 = h(C_i \| N_1 \| N_2 \| T_1) \quad (5)$$

$$M_2 = (C_i, N_2, \beta_1, T_2) \quad (6)$$

Format: 128 + 128 + 160 + 32 = 448 bits.

Message 3 (PUF Response): UAV_{*i*} $\rightarrow$ GS

- 1) Evaluate PUF: $R_i^{\text{noisy}} = \text{PUF}_i(C_i)$
 - 2) BCH correction: $R_i = \text{BCH_correct}(R_i^{\text{noisy}}, h_i)$
 - 3) Mask response: $R_i^{\text{masked}} = R_i \oplus h(N_2)$
 - 4) Auth token: $\gamma_1 = h(R_i \| N_1 \| N_2 \| T_2)$
- $$M_3 = (R_i^{\text{masked}}, \gamma_1, C'_i, T_3) \quad (7)$$

Format: 128 + 160 + 128 + 32 = 448 bits.

Message 4 (Authentication Confirmation): GS $\rightarrow$ UAV_{*i*}

The GS unmask the response, verifies via BCH decoding and Hamming distance check, then distributes encrypted peer credentials and a confirmation MAC:

$$\text{SK}_{i,\text{GS}} = h(R_i \| N_1 \| N_2) \quad (8)$$

$$\delta_1 = h(\text{SK}_{i,\text{GS}} \| \text{Cred}_i^{\text{enc}} \| T_4) \quad (9)$$

$$M_4 = (\text{Cred}_i^{\text{enc}}, \delta_1, \text{TID}_i^{\text{new}}, T_4) \quad (10)$$

Format: ≈ 400 bits. Total Phase 2: 1680 bits $\approx$ 210 bytes.

D. Phase 3: UAV-to-UAV Peer Authentication

Phase 3 enables direct mutual authentication between UAV pairs using credentials distributed in Phase 2. This three-message protocol requires no GS involvement.

Message P1 (Peer Auth Request): UAV_{*i*} $\rightarrow$ UAV_{*j*}

$$\eta_1 = h(\text{TID}_i \| \text{TID}_j \| n_i \| T_{p1}) \quad (11)$$

$$P_1 = (\text{TID}_i, C_j, n_i, \eta_1, T_{p1}) \quad (12)$$

Format: 64 + 128 + 160 + 160 + 32 = 544 bits.

Message P2 (Challenge Response): UAV_{*j*} $\rightarrow$ UAV_{*i*}

UAV_{*j*} generates its PUF response, reconstructs peer secrets via XOR masking, creates an encrypted authentication puzzle Q_i , and computes a response MAC:

$$P_{ji} = h(R_j \| \text{ID}_i \| N_{\text{gs}}) \quad (13)$$

$$\tilde{P}_{ji} = \text{Mask}_{ji} \oplus P_{ji} \quad (14)$$

$$Q_i = E_{\tilde{P}_{ji}}(n_j \| C_i \| T_{p1}) \quad (15)$$

$$\eta_2 = h(Q_i \| n_i \| T_{p2}) \quad (16)$$

$$P_2 = (Q_i, \eta_2, T_{p2}) \quad (17)$$

Format: 320 + 160 + 32 = 512 bits.

Message P3 (Mutual Auth Completion): UAV_{*i*} $\rightarrow$ UAV_{*j*}

UAV_{*i*} verifies the puzzle, reconstructs UAV_{*j*}'s secret, and creates a reciprocal puzzle:

$$Q_j = E_{\tilde{P}_{ji}}(n'_i \| T_{p2}) \quad (18)$$

$$\eta_3 = h(Q_j \| n_j \| T_{p3}) \quad (19)$$

$$P_3 = (Q_j, \eta_3, T_{p3}) \quad (20)$$

Format: 320 + 160 + 32 = 512 bits. Total Phase 3: 1568 bits $\approx$ 196 bytes.

For a swarm of N UAVs, full-mesh authentication requires $N(N-1)/2$ Phase 3 executions (45 for $N=10$).

TABLE II: Simulation Parameters

Parameter	Value
Simulation area	500 $\times$ 300 m (stadium)
Number of UAVs	10 (perimeter positions)
Ground station	Center (275, 175 m)
Link bitrate	6 Mbps (802.11a)
Propagation speed	3×10^8 m/s
Processing delay	0.1 ms mean
Jitter std. dev.	0.02 ms
PUF type	Arbiter PUF (128 stages)
PUF noise	3% BER
CRPs per UAV	12
BCH code	(255, 131, 18)
Hash output	160 bits
Nonce size	128 bits
Simulation time	120 s

E. Phase 4: Session Key Establishment

Both UAVs compute a shared symmetric session key:

$$\text{SK}_{ij} = h(n_i \| n_j \| n'_i \| R_i \| R_j) \quad (21)$$

Properties: 160-bit key, forward secrecy via ephemeral nonces, unique per session, and cannot be derived without PUF responses.

IV. OMNET++ IMPLEMENTATION

A. Simulation Environment

We implement the protocol in OMNeT++ 6.3.0 using C++ with clang release-mode compilation (`-O3`). The simulation models a stadium aerial surveillance scenario with the following parameters:

B. Module Architecture

The OMNeT++ implementation consists of the following modules:

- **UAVNode:** Simple module handling Phase 2 and Phase 3 protocol flows, PUF evaluation, and per-peer metric collection.
- **GroundStation:** Manages enrollment database, Phase 2 challenge-response verification, BCH error correction, and peer credential distribution.
- **NetworkController:** Periodic status reporting.

Communication uses OMNeT++'s `sendDirect()` with a delay model comprising propagation delay (distance/speed of light), serialization delay (bits/bitrate), processing delay, and random jitter.

C. Cryptographic Components

SHA3-256: Implemented via OpenSSL's EVP API, truncated to 160 bits for protocol compatibility. Benefits from hand-optimized Keccak assembly in OpenSSL.

SPONGENT-160: Custom C++ implementation with 16-bit rate, 144-bit capacity, and 80 permutation rounds. Includes the standard 4-bit S-box layer and bit-level permutation layer ($\pi(i) = 40i \bmod (n-1)$).

PUF Simulator: Arbiter PUF model with 128 delay stages, Gaussian noise addition, and configurable bit error rate.

TABLE III: Phase 2 Authentication Results (Mean of 10 UAVs)

Metric	SHA3	SPONGENT
Total latency (ms)	0.672	7.368
Compute time (ms)	0.007	6.703
Hash time (ms)	0.003	6.697
Network delay (ms)	0.665	0.665
Overhead (bytes)	175	175
Auth success rate	100%	100%

BCH Codec: BCH(255, 131, 18) with Galois field $GF(2^8)$ tables, Berlekamp-Massey error locator polynomial, and Chien search. Helper data follows the fuzzy extractor scheme of Eq. (2).

D. Full-Mesh Phase 3 Implementation

Each UAV initiates Phase 3 with all higher-ID UAVs, yielding $N(N - 1)/2 = 45$ unique pairs. Per-peer metrics are recorded as separate OMNeT++ scalars (phase3_peer_X_*), and responder-side metrics (phase3_resp_X_*) are cross-referenced during CSV export to produce complete per-pair timing breakdowns.

E. Hash Warmup

OpenSSL's first SHA3 invocation performs lazy algorithm loading (~ 2.8 ms for `EVP_MD_fetch`). We add a warmup hash call during `initialize()` to absorb this one-time cost before protocol timing begins, ensuring consistent measurements across all UAVs.

V. SIMULATION RESULTS

A. Phase 2: UAV-GS Authentication

Table III summarizes Phase 2 performance across both hash modes for all 10 UAVs.

Key observations:

- SHA3 compute is dominated by non-hash overhead (PUF evaluation, BCH, memory operations); hash itself is only $\sim 3.4 \mu\text{s}$ total across ~ 8 calls ($\sim 0.4 \mu\text{s}/\text{call}$).
- SPONGENT compute is 99.9% hash time, at $\sim 890 \mu\text{s}/\text{call}$ due to the 80-round bit-level permutation in software.
- Network delay is identical across hash modes (same message sizes), comprising propagation ($\sim 1.7 \mu\text{s}$ at 500 m), serialization (~ 0.23 ms/message at 6 Mbps), and processing/jitter.

B. Phase 2 Per-Message Breakdown

Table IV shows the detailed timing for each of the four messages.

C. Phase 3: Peer-to-Peer Authentication

Table V summarizes Phase 3 performance across all 45 full-mesh pairs.

Phase 3 SPONGENT compute (~ 14 ms) is $\sim 2\times$ Phase 2 (~ 7 ms) because the three-message handshake involves hash operations on *both* the initiator and responder UAVs: approximately 8 hash calls on each side (build token, verify token, derive session key) totaling ~ 16 calls per pair.

TABLE IV: Phase 2 Per-Message Timing (SHA3 Mode, Mean)

Step	Compute (μs)	Network (μs)
M1: UAV auth hash	3.0	166
M1: GS processing	5.5	—
M2: UAV verify MAC	2.1	166
M2: PUF evaluation	2.8	—
M2: BCH correction	22.0	—
M3: Crypto (mask+token)	4.5	166
M3: GS verify+BCH	53.0	—
M4: UAV finalize	0.1	166

TABLE V: Phase 3 Peer Authentication Results (Mean of 45 Pairs)

Metric	SHA3	SPONGENT
Total latency (ms)	0.476	14.535
Compute time (ms)	0.009	14.069
Hash time (ms)	0.006	14.062
Network delay (ms)	0.467	0.467
Overhead (bytes)	104	104
Success rate	45/45	45/45

D. Phase 3 Per-Step Breakdown

Table VI decomposes Phase 3 into its three message exchange steps.

E. Scalability: Full-Mesh Results

All 45 Phase 3 pairs complete within a narrow latency band: SHA3 range [0.46, 0.50] ms; SPONGENT range [13.9, 15.2] ms. The variance is primarily due to random jitter in the link delay model. No pair failures occurred.

For a swarm of N UAVs, the total Phase 3 authentication work scales as $O(N^2)$ pairs but can be parallelized since OMNeT++'s discrete event model allows overlapping message exchanges.

VI. COMPARISON WITH EXISTING METHODS

A. Communication Overhead

Table VII compares the communication overhead of our protocol with traditional authentication methods.

Our protocol's 430 bytes total (Phase 2: 210B + Phase 3: 196B + Phase 4: 24B) is competitive with ECC-256 while providing PUF-based physical security and no key storage requirement.

B. Authentication Latency

Table VIII compares authentication latency across methods. Theoretical values from the literature are shown alongside our measured OMNeT++ results.

Our SHA3-based Phase 2 authentication (0.67 ms) is $12\text{--}22\times$ faster than RSA/ECC and competitive with pre-shared key approaches, while providing stronger security guarantees (no stored keys, PUF unclonability).

C. Computational Cost

Table IX breaks down computational operations per phase.

TABLE VI: Phase 3 Per-Step Timing (SHA3 Mode, Mean of 45 Pairs)

Step	Compute (μ s)	Network (μ s)
P1: Initiator build request	3.2	155
P1-P2: Responder process	5.0	—
P2: Network transit	—	155
P2-P3: Initiator verify+build	7.5	—
P3: Responder verify	4.0	155
Total	19.7	465

TABLE VII: Communication Overhead Comparison

Method	Bytes	Messages
PUF 4-phase (this work)	430	8
RSA-2048 mutual auth	768	4
ECC-256 mutual auth	384	4
Pre-shared key (AES-GCM)	256	3
PUF [2]	512	6
PUF [12]	640	8

D. Storage Requirements

Per UAV in a swarm of N UAVs:

- Own identity (ID, TID): 16 bytes
- Peer credentials ($N - 1$ peers $\times$ 40 bytes): $40(N - 1)$ bytes
- Helper data (12 CRPs $\times$ 32 bytes): 384 bytes
- Active session keys ($\sim 5 \times 32$ bytes): 160 bytes

Total: $560 + 40N$ bytes. For $N = 10$: ~ 960 bytes ≈ 1 KB.

Ground Station: ~ 1 KB/UAV. For 100-UAV fleet: ~ 100 KB.

VII. HARDWARE IMPLEMENTATION ANALYSIS

A. Software vs. Hardware Performance

Our OMNeT++ simulation runs cryptographic primitives in software on an x86-64 processor. Table X compares measured software timing with projected hardware performance.

Key insight: In software, SPONGENT is $2225\times$ slower than SHA3 because its bit-level permutation layer processes individual bits—catastrophic for 64-bit CPUs. In hardware, SPONGENT is only $2\text{--}5\times$ slower because the permutation is just wire routing (zero logic delay) and the S-box layer operates on all nibbles in parallel.

B. Hardware Area and Power

SPONGENT-160 requires only 2,190 GE—suitable for implementation alongside a PUF on a constrained UAV SoC where area and power budget are critical. With hardware SPONGENT, Phase 2 total compute would drop from 7 ms to $\sim 50\text{--}100$ μ s, and Phase 3 from 14 ms to $\sim 100\text{--}200$ μ s.

C. Projected Hardware Protocol Latency

VIII. SECURITY ANALYSIS

A. Phase 2: Mutual Authentication

Theorem 1 (Mutual Authentication). *After Phase 2, both UAV_i and GS are mutually authenticated.*

Proof: (1) UAV_i is authenticated by GS via correct PUF response R_i —only the authentic UAV with the physical PUF

TABLE VIII: Authentication Latency Comparison

Method	Latency	Platform
PUF-SHA3 (this work)	0.67 ms	OMNeT++ (x86)
PUF-SPONGENT (this work)	7.37 ms	OMNeT++ (x86)
PUF-SPONGENT (FPGA est.)	~ 0.1 ms	FPGA projected
PUF [2]	10–50 ms	Raspberry Pi 4B
RSA-2048	~ 15 ms	ARM Cortex-A72
ECC-256	~ 8 ms	ARM Cortex-A72
Pre-shared key	~ 0.5 ms	ARM Cortex-A72

TABLE IX: Computational Operations per Authentication Phase

Operation	Phase 2	Phase 3	Phase 4
Hash calls	~ 8	~ 16	1
PUF evaluations	1	0	0
BCH decode	1	0	0
XOR (128-bit)	2	4	0
AES encrypt/decrypt	0	2	0
ECDH (optional)	0	0	2

can generate this. (2) GS is authenticated by UAV_i via the correct challenge MAC β_1 computed from the CRP database that only GS possesses. (3) An adversary cannot forge R_i without physical access to the PUF (unclonability property). (4) An adversary cannot predict R_i from C_i without the PUF (unpredictability property). $\square$

B. Replay Attack Resistance

Theorem 2 (Replay Resistance). *Phase 2 prevents replay attacks.*

Proof: (1) Fresh timestamps T_1, T_2, T_3, T_4 in all messages. (2) Fresh nonces N_1, N_2 ensure message uniqueness. (3) Messages with $|T_{\text{current}} - T_i| > \Delta T_{\text{max}}$ are rejected. (4) CRPs are marked “consumed” after use, preventing challenge reuse. $\square$

C. Phase 3: Peer Mutual Authentication

Theorem 3 (Peer Mutual Authentication). *After Phase 3, UAV_i and UAV_j mutually authenticate without GS involvement.*

Proof: (1) UAV_j is authenticated by UAV_i via correctly decrypting puzzle Q_i , which requires knowledge of P_{ij} derivable only from UAV_j’s PUF response. (2) Symmetrically, UAV_i is authenticated via Q_j . (3) The XOR masks $\text{Mask}_{ij} = P_{ij} \oplus P_{ji}$ are information-theoretically secure one-time pads. (4) An adversary observing Mask_{ij} cannot extract P_{ij} or P_{ji} without PUF responses. $\square$

D. Man-in-the-Middle Resistance

Theorem 4 (MITM Resistance). *The protocol prevents man-in-the-middle attacks.*

Proof: (1) Encrypted puzzles Q_i, Q_j cannot be modified without detection (MAC-protected via η_2, η_3). (2) Puzzles cannot be decrypted without PUF-derived secrets P_{ij}, P_{ji} . (3) Forging valid puzzles requires knowledge of secrets derivable only from PUF responses. (4) Any modification is detected via MAC verification. $\square$

TABLE X: Software vs. Hardware Hash Performance

Hash	Software (x86)	FPGA (100 MHz)	ASIC (200 MHz)
SHA3-256	0.4 μ s	1–2 μ s	0.5–1 μ s
SPONGENT-160	890 μ s	5–10 μ s	2–5 μ s
Ratio	2225 $\times$	2–5 $\times$	2–5 $\times$

TABLE XI: Hardware Resource Comparison

Component	Gate Count (GE)	Power (μ W)	Latency (μ s)
SPONGENT-160	2,190	3–5	5–10
SHA3/Keccak	38,000	50–100	1–5
AES-128	11,000	20–30	1
Arbiter PUF	1,200	0.5	0.03
BCH(255,131,18)	5,000	10	2

E. Key Secrecy and Forward Secrecy

Theorem 5 (Key Secrecy). *Session key SK_{ij} is known only to UAV_i and UAV_j.*

Proof: $SK_{ij} = h(n_i || n_j || n'_i || R_i || R_j)$. Nonces are ephemeral and transmitted only encrypted or as hash inputs. PUF responses are never transmitted in clear (masked with hash output). Without PUF access, an adversary cannot derive the key. Hash pre-image resistance prevents recovery from observed values. $\square$

With the optional ECDH extension in Phase 4, the protocol achieves perfect forward secrecy: even if PUF responses R_i, R_j are compromised later, past session keys remain secure due to the ephemeral Diffie-Hellman contribution.

IX. DISCUSSION

A. Simulation vs. Theory Overhead

Our OMNeT++ implementation measures 175 bytes for Phase 2 and 104 bytes for Phase 3, while the theoretical protocol specifies 210 bytes and 196 bytes respectively. This difference arises because:

- 1) The simulation omits the encrypted credential package in Message 4 (simplified to a raw network credential).
- 2) Phase 3 messages do not include the full TID_i field (8 bytes) and the encrypted puzzle Q_i (40 bytes) specified in theory.
- 3) The simulation counts only protocol-level payload, excluding message framing headers.

The theoretical values represent the complete deployed protocol; our simulation accurately times the computational core while slightly underestimating communication overhead. The timing impact is <0.1 ms at 6 Mbps.

B. PUF Noise and BCH Effectiveness

With 3% PUF noise and BCH(255, 131, 18) correcting up to 18 bit errors ($\sim 14\%$ of 128 bits), our simulation achieves 100% authentication success. The BCH helper data follows the fuzzy extractor scheme $h_i = \text{BCH_encode}(R_i) \oplus R_i$, ensuring that helper data is public-safe (does not leak PUF responses).

TABLE XII: Projected Protocol Latency on Hardware Platforms

Phase	Software	FPGA	ASIC
Phase 2 (SHA3)	0.67 ms	20 μ s	10 μ s
Phase 2 (SPONGENT)	7.37 ms	80 μ s	40 μ s
Phase 3 (SHA3)	0.48 ms	35 μ s	15 μ s
Phase 3 (SPONGENT)	14.54 ms	160 μ s	80 μ s
Phase 4 (basic)	<0.01 ms	5 μ s	3 μ s
Total (SPONGENT)	21.91 ms	245 μs	123 μs

C. Network Delay Analysis

The measured network delay (~ 0.66 ms Phase 2, ~ 0.47 ms Phase 3) reflects the simulation's `sendDirect()` model, which includes propagation, serialization, and processing delays but not MAC-layer contention (CSMA/CA back-off, RTS/CTS). In real deployments with 802.11ac WiFi, MAC overhead would add 1–10 ms per frame, increasing total latency by 4–40 ms for Phase 2's four messages.

D. Swarm Scalability

Full-mesh Phase 3 requires $N(N-1)/2$ pair authentications. For $N = 10$, this yields 45 pairs; for $N = 20$, 190 pairs. Since Phase 3 exchanges are independent, they can be parallelized across different UAV pairs. In our simulation, all 45 pairs complete within the 120-second simulation window with staggered start times.

X. CONCLUSION

We presented a four-phase PUF-based authentication protocol for UAV swarm networks, providing mutual UAV-GS authentication, direct peer-to-peer UAV authentication without Ground Station involvement, and secure session key establishment. Our OMNeT++ simulation with 10 UAVs in a stadium surveillance scenario demonstrates:

- **Fast authentication:** 0.67 ms (SHA3) / 7.37 ms (SPONGENT) for Phase 2; 0.48 ms / 14.54 ms for Phase 3.
- **100% success:** All 10 UAV-GS authentications and all 45 peer pairs succeed.
- **Lightweight:** 430 bytes total communication, only hash/XOR/PUF operations.
- **Scalable:** $O(N^2)$ peer pairs but parallelizable; 1 KB storage per UAV.
- **Hardware-ready:** SPONGENT-160 at 2,190 GE enables projected sub-0.25 ms total auth on FPGA, sub-0.12 ms on ASIC.

The protocol achieves 10–18 $\times$ faster authentication than RSA-2048 or ECC-256 while providing PUF-based physical unclonability, replay resistance, MITM resistance, and optional perfect forward secrecy.

Future work includes: (1) Implementation on Xilinx Artix-7 FPGA with hardware SPONGENT and Arbiter PUF, (2) integration with MAVLink/ROS flight controllers, (3) evaluation under adversarial scenarios (jamming, Sybil attacks), and (4) scalability testing with 50–100 UAV swarms.

ACKNOWLEDGMENT

The authors thank the OMNeT++ community for the simulation framework and the OpenSSL project for the SHA3 implementation.

REFERENCES

- [1] A. Mukherjee *et al.*, “Securing UAV Flying Ad Hoc Wireless Networks: Authentication Development for Robust Communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Communications*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera *et al.*, “Secure and Efficient Peer2Peer Authentication-Attestation Protocol for UAV Networks,” in *Proc. IEEE INFOCOM Workshop*, 2023, pp. 1–6.
- [4] National Technological University, “PUF-Based Lightweight Security Framework for Drone Networks,” DR-NTU Technical Report, 2023.
- [5] S. Dey and M. Hossain, “MITM- and DoS-Resistant PUF Authentication for Industrial WSNs via Sensor-Initiated Registration,” *IEEE Internet of Things J.*, vol. 10, no. 22, pp. 19740–19753, 2023.
- [6] J. Delvaux, “Cryptographic Key Generation from PUF Data Using Efficient Fuzzy Extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [7] L. Zhang *et al.*, “SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023.
- [8] R. Maes, “Helper Data Masking for Physically Unclonable Function-Based Key Generation,” *Cryptology ePrint Archive*, Report 2019/688, 2019.
- [9] U. Chatterjee *et al.*, “PUF-based Authentication in IoT against Strong Physical Adversary,” in *Proc. IEEE DATE*, 2022, pp. 1–6.
- [10] M. Hassan *et al.*, “Designing secure PUF-based authentication protocols for IoT devices,” *Scientific Reports*, vol. 13, p. 21856, 2023.
- [11] S. Banerjee *et al.*, “On the Security of the Novel Authentication Scheme for UAV-Ground Station Communication,” in *Proc. IEEE WiMob*, 2023, pp. 1–6.
- [12] B. Li *et al.*, “PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [13] A. Alkathheiri *et al.*, “Review of Physically Unclonable Functions (PUFs): Structures, Models, and Algorithms,” *Frontiers in Electronics*, vol. 4, p. 1123456, 2023.
- [14] R. Aman *et al.*, “PUF based Lightweight Authentication and Key Exchange Protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [15] M. Rose *et al.*, “mrPUF: A Memristive Device based Physical Unclonable Function,” *IEEE Trans. VLSI Syst.*, vol. 31, no. 6, pp. 901–910, 2023.
- [16] D. Lim, “Physical Unclonable Functions (PUF),” *IEEE Solid-State Circuits Magazine*, vol. 15, no. 2, pp. 42–53, 2023.
- [17] S. Khan *et al.*, “A Probably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks,” *IEEE Internet of Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [18] Y. Gao *et al.*, “A dependable and resource-frugal ring oscillator physically unclonable function,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 9, pp. 3012–3023, 2023.
- [19] S. Kumar *et al.*, “Design of A Ring Oscillator Based PUF with Enhanced Challenge Response pair and Improved Reliability,” *IEEE Access*, vol. 11, pp. 75432–75445, 2023.
- [20] T. Eisenbarth *et al.*, “Design and Evaluation of Lightweight Cryptographic Primitives for Constrained Devices,” *J. Cryptographic Engineering*, vol. 12, no. 3, pp. 245–260, 2022.
- [21] L. Xu *et al.*, “An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices,” *IEEE Internet of Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [22] A. Bogdanov *et al.*, “SPONGENT: A Lightweight Hash Function,” in *Proc. CHES*, Springer LNCS, vol. 6917, pp. 312–325, 2011.
- [23] NIST, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, Aug. 2015.